Report

Presidency University RAG Chatbot Using LangChain, FAISS, Hugging Face, Groq, and FastAPI

Table of Contents

1. Introduction

Artificial Intelligence has transformed the way users interact with information. Traditional chatbots rely only on the knowledge stored within a Large Language Model (LLM), which may produce inaccurate or fabricated responses, commonly known as hallucinations.

To overcome this limitation, Retrieval-Augmented Generation (RAG) combines document retrieval with language generation. Before answering a user's query, the chatbot searches a knowledge base for relevant information and supplies that information to the language model. This enables the chatbot to generate accurate, context-aware responses.

This project implements a **Presidency University RAG Chatbot** that answers questions based on a PDF containing information about Presidency University. The chatbot uses LangChain for orchestration, FAISS for vector similarity search, Hugging Face embeddings for semantic representation, Groq's Llama model for response generation, FastAPI for backend services, and an HTML/CSS frontend for user interaction.

2. Problem Statement

Students, parents, and visitors often need information regarding admissions, courses, campus facilities, rankings, placements, and university policies. Searching through lengthy PDF documents is time-consuming and inconvenient.

The objective of this project is to develop an AI chatbot capable of retrieving relevant information from the university document and providing accurate answers in natural language.

3. Objectives

The objectives of this project are:

Develop an intelligent chatbot using Retrieval-Augmented Generation (RAG).

Extract information from a PDF document.

Convert document text into semantic vector embeddings.

Store embeddings in a FAISS vector database.

Retrieve relevant information based on user queries.

Generate context-aware responses using a Large Language Model.

Build an interactive web interface using HTML and CSS.

Expose chatbot functionality through FastAPI APIs.

4. Technologies Used

Technology	Purpose
Python	Core programming language
LangChain	RAG pipeline orchestration
PyPDFLoader	PDF document loading
RecursiveCharacterTextSplitter	Document chunking
Hugging Face Embeddings	Semantic embedding generation
FAISS	Vector database
Groq API	Llama 3.1 language model
FastAPI	Backend REST API
HTML	User interface
CSS	Frontend styling
JavaScript	Communication between frontend and backend

5. Methodology

The chatbot follows the Retrieval-Augmented Generation (RAG) workflow.

Step 1: PDF Loading

The university PDF document is loaded using the PyPDFLoader provided by LangChain Community. Each page is extracted as a document object.

Step 2: Text Splitting

The extracted text is divided into smaller chunks using RecursiveCharacterTextSplitter. Chunking improves retrieval efficiency and ensures that the embedding model processes manageable portions of text.

Step 3: Embedding Generation

Each chunk is converted into a dense numerical vector using the Hugging Face model **sentence-transformers/all-MiniLM-L6-v2**. These embeddings capture the semantic meaning of the text.

Step 4: Vector Database Creation

The generated embeddings are stored in a FAISS vector database. FAISS enables efficient similarity search among thousands of text chunks.

Step 5: User Query

The user enters a question through the web interface.

Step 6: Retrieval

The user's question is embedded using the same embedding model. FAISS compares the query vector with stored vectors and retrieves the most relevant document chunks.

Step 7: Response Generation

The retrieved context and the user's question are sent to the Groq-hosted Llama 3.1 language model. The model generates an accurate response based only on the retrieved information.

Step 8: Display

The generated answer is returned through FastAPI and displayed in the chatbot interface.

6. Project Workflow

University PDF
Load PDF
Split into Chunks
Generate Embeddings
Store in FAISS
User Question
Retrieve Similar Chunks
Groq LLM
Generate Answer
Display Response

7. Implementation

The implementation consists of the following modules:

loader.py

Loads the university PDF.

Extracts document pages.

create_index.py

Splits text into chunks.

Generates embeddings.

Creates the FAISS vector index.

Saves the index for future retrieval.

rag.py

Loads the saved FAISS index.

Retrieves relevant document chunks.

Creates the prompt.

Sends context to the Groq LLM.

Returns the generated response.

main.py

Implements the FastAPI backend.

Defines REST API endpoints.

Connects the frontend with the RAG pipeline.

index.html

Provides a clean chatbot interface.

Sends user questions using JavaScript.

Displays responses returned by the backend.

8. API Design

GET /

Returns the API status.

Example Response:


```
{
   "message":"Presidency RAG Chatbot API Running"
}
```


GET /home

Loads the chatbot web interface.

POST /chat

Accepts user questions.

Example Request:

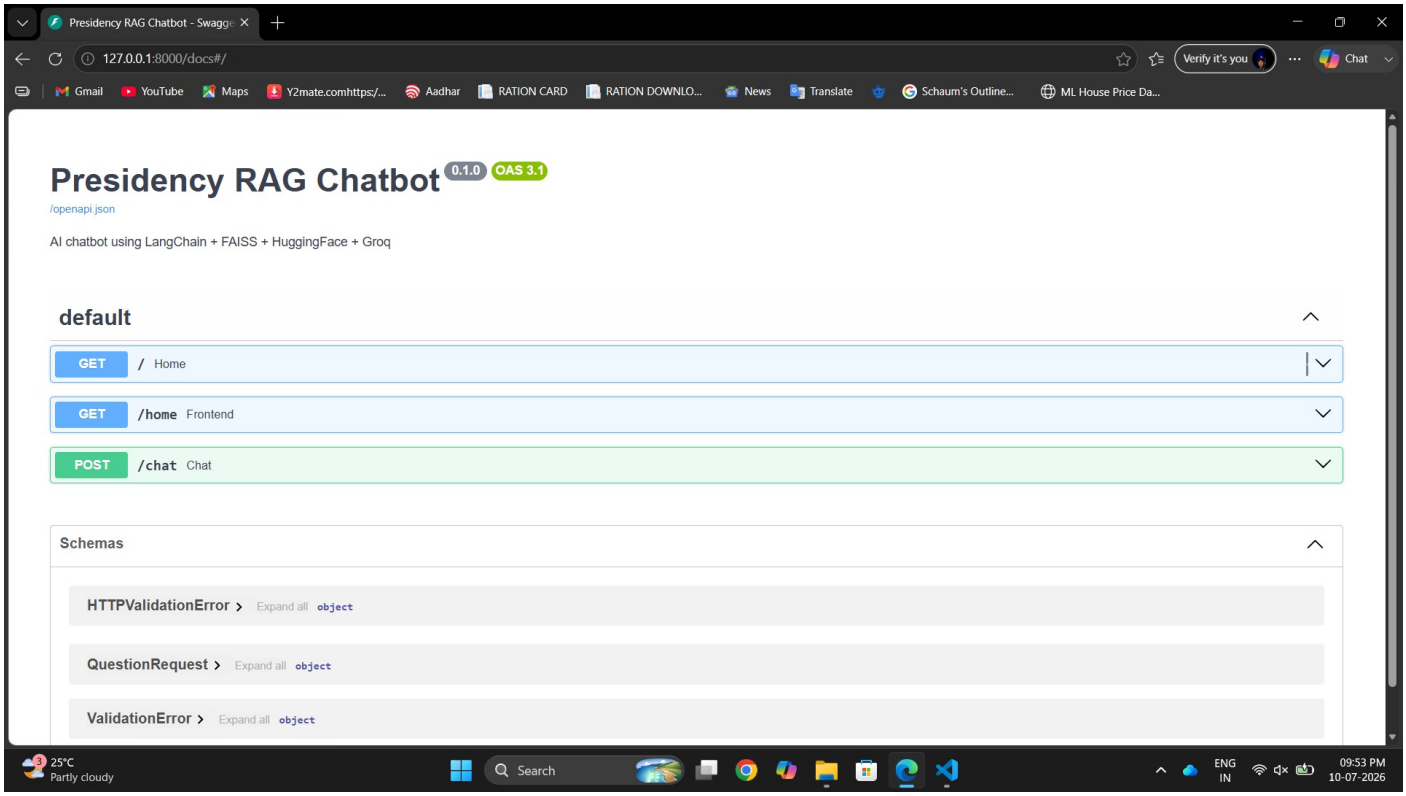

```
{
   "question":"When was Presidency University established?"
}
```


Example Response:


```
{
   "question":"When was Presidency University established?",
   "answer":"Presidency University was established in 2013 under the Presidency University Act, Karnataka State Act No. 41."
```

}

Output:



9. Testing and Result

The project was tested at every stage.

PDF Loading

Successfully loaded the university PDF.

Total pages processed: **29**

Text Chunking

The document was divided into **61 text chunks**.

Embedding Generation

Successfully generated embeddings using **all-MiniLM-L6-v2**.

FAISS Index

FAISS vector database created successfully.

Generated files:

index.faiss

index.pkl

RAG Retrieval

Sample Question:

When was Presidency University established?

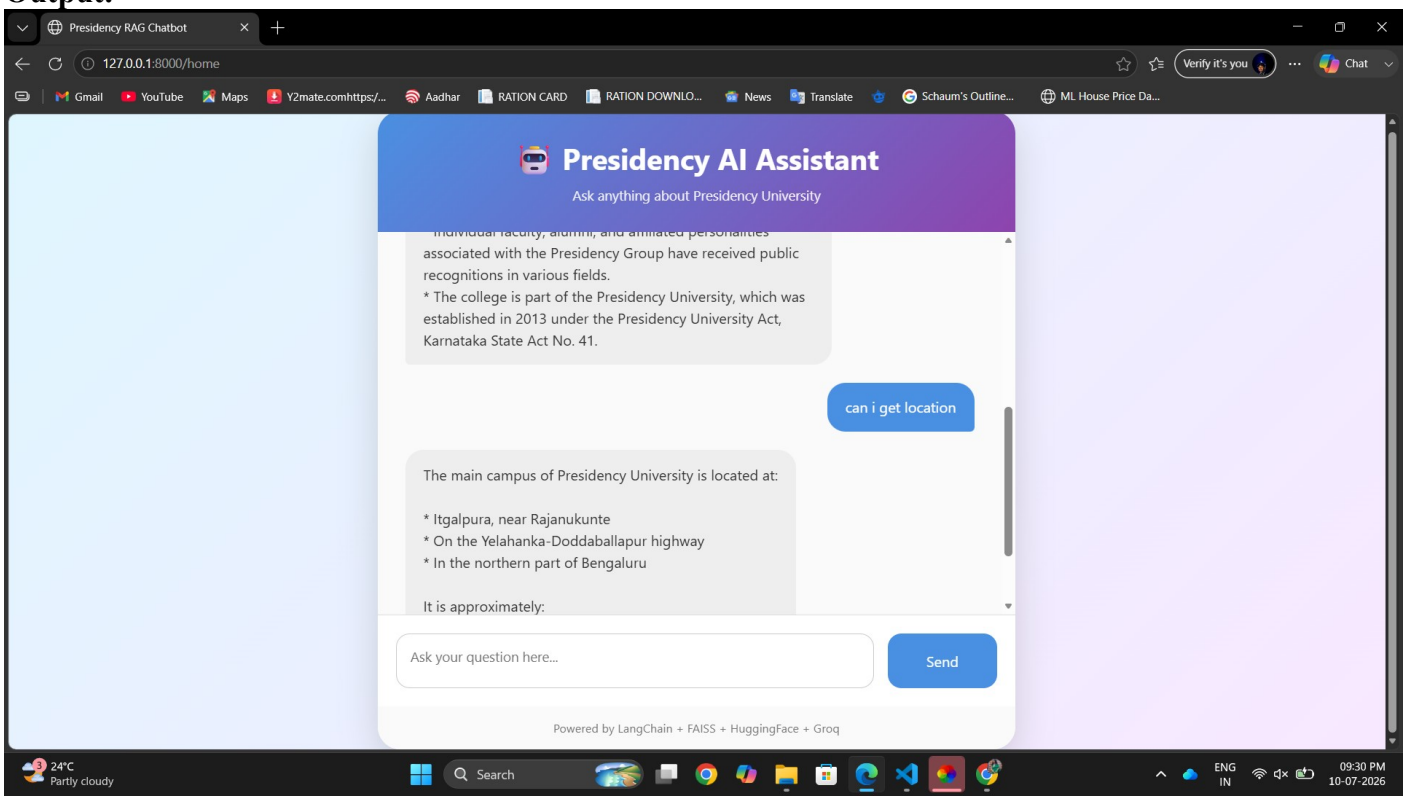
Generated Answer:

"Presidency University was established in 2013 under the Presidency University Act, Karnataka State Act No. 41."

FastAPI

The API successfully accepted user queries and returned responses through the chatbot interface.

Output:



10. Advantages

Provides accurate document-based answers.

Reduces hallucinations.

Fast semantic search.

Easy to update by replacing the PDF.

Modular architecture.

Interactive web interface.

Suitable for educational institutions.

Scalable for larger document collections.

11. Limitations

Limited to the uploaded document.

Requires rebuilding the vector index after document updates.

Depends on internet connectivity for Groq API access.

Cannot answer questions outside the available document.

12. Applications

University information assistant

Admission support chatbot

Student helpdesk

Digital library assistant

Institutional knowledge management

Educational document search system

13. Future Enhancements

Support multiple PDF documents.

Display source page numbers with answers.

Add voice input and speech output.

Implement multilingual support.

Store chat history in a database.

User authentication.

Admin portal for document management.

Cloud deployment.

Mobile-friendly interface.

Integration with university websites.

14. Conclusion

The Presidency University RAG Chatbot demonstrates the practical implementation of Retrieval-Augmented Generation using modern AI technologies. The system combines LangChain, Hugging Face embeddings, FAISS, Groq Llama 3.1, FastAPI, and a web-based frontend to provide reliable and context-aware answers from a university PDF document. By retrieving relevant information before generating responses, the chatbot minimizes hallucinations and improves answer accuracy. The project illustrates how RAG can be effectively applied to educational institutions for intelligent document-based assistance and can be extended to other domains with minimal modifications.